CryptoPulse

Real-Time Multi-Asset Crypto Pipeline & Analytics

End-to-End Big Data Architecture

Awais Waheed Sandesh Shahi Aung Thu Moe

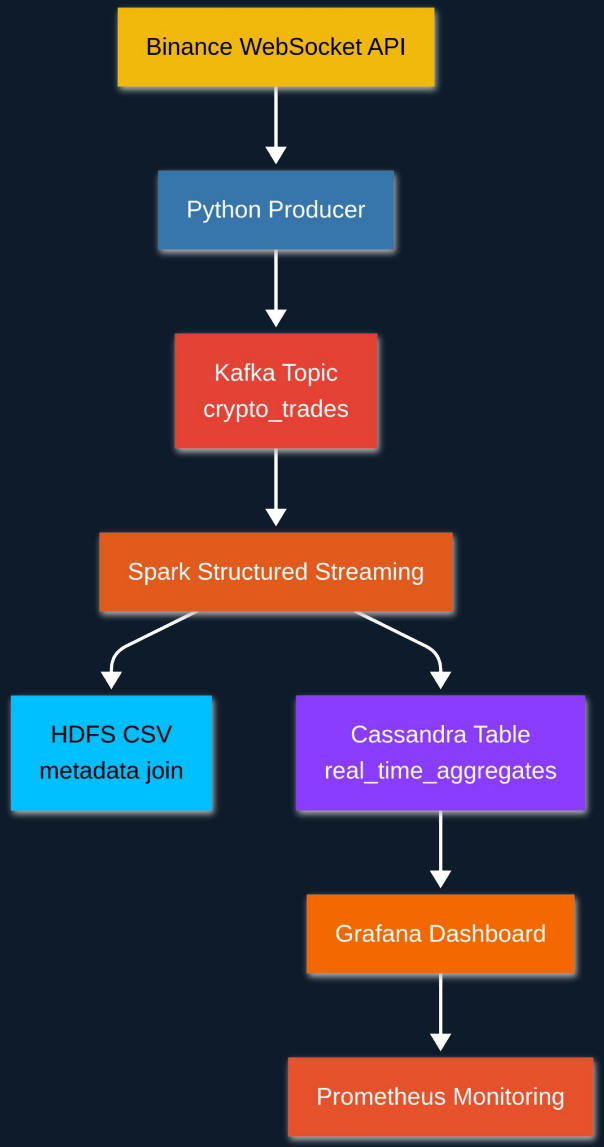
Big Data Systems — University Project

The Problem

- Crypto exchanges emit thousands of raw trade events per second across multiple assets
- A single event — one price, one timestamp — is not actionable on its own
- Three requirements for real value:
  - Time-windowed aggregations (average price, total volume per minute)
  - Human-readable enrichment (BTCUSDT ⟶ Bitcoin, Layer 1)
  - Real-time anomaly detection (flag volatility spikes without holding full history in memory)
- Challenge: joining a live stream with static reference data while maintaining fault tolerance
- Goal: a containerised, production-grade pipeline that handles all of this end-to-end

6 Assets · Thousands of Events/sec · Zero Manual Intervention

System Architecture



7 Docker services
Shared big-data-net bridge network

6 crypto assets
Single multiplexed WebSocket connection

Event-time watermarking
Handles late-arriving data

Auto-provisioned
Grafana + Prometheus, zero manual setup

Why Cassandra Over HBase?

Criterion	HBase	Cassandra
Architecture	Master/slave (HMaster + RegionServers)	Masterless peer-to-peer ring
Consistency	Strong (CP)	Tunable (eventual by default)
Write Profile	Good	Excellent — LSM tree optimised for writes
Time-Series Fit	Possible but not native	Native — partition/clustering key model
Spark Integration	HBase-Spark connector (less mature)	spark-cassandra-connector (first-class)
Query Language	Java API / HBase Shell	CQL — SQL-like, familiar
Single-Node Setup	Complex — requires HDFS + ZooKeeper	Trivial single Docker container
Grafana Plugin	Not available	hadesarchitect-cassandra-datasource

Verdict: Write-heavy workload + time-series schema + first-class Spark connector = Cassandra wins

Layer 1 — Real-Time Ingestion

Apache Kafka + Python WebSocket Producer

- Binance combined WebSocket stream — 6 assets over a single persistent connection
  - BTC · ETH · SOL · BNB · ADA · XRP
- websocket-client with 20-second ping keepalive — auto-reconnects on transient drops
- kafka-python producer serialises each event to JSON and publishes to crypto_trades
- Topic provisioned with 3 partitions — Spark consumes in parallel across executors
- Decoupled modules: kafka_client.py (broker logic) vs binance_producer.py (WebSocket logic)

```
# Combined stream – 6 assets, 1
connection
ws = websocket.WebSocketApp(
    BINANCE_WS_URL,
    on_message=on_message,
    on_error=on_error,
    on_close=on_close
)
ws.run_forever(
    ping_interval=20,
    ping_timeout=10
)
```

Layer 2 — Stream Processing

Apache Spark 3.5 Structured Streaming

- Subscribes to Kafka topic; null/malformed messages filtered before aggregation
- 1-minute tumbling windows with 30-second watermark — tolerates late-arriving events
- Per-window, per-asset aggregations:
  - avg_price · total_volume · high_price · low_price
- Stream-static join with HDFS CSV — BTCUSDT → "Bitcoin", "Layer 1"
- Anomaly detection: is_anomaly = True if (high − low) / low > 1.5% in any 60-second window
- foreachBatch sink — empty batches skipped; valid batches bulk-written to Cassandra

```
aggregated_df = cleaned_df \
  .withWatermark("trade_timestamp",
                 "1 minute") \
  .groupBy(

window(col("trade_timestamp"),
             "1 minute"),
      col("s").alias("symbol")
  ).agg(

avg("price").alias("average_price"),
      spark_sum("quantity")
          .alias("total_volume"),
```

Layers 3 & 4 — Storage & Infrastructure

Cassandra Schema

- PRIMARY KEY (window_start, symbol)
- Partitions by time window — single partition scan per Grafana query
- Data persisted in Docker volumes — survives container restarts

HDFS

- Serves crypto_metadata.csv — 6-row static reference table
- Read once at Spark startup and cached — zero repeated HDFS calls

Docker Compose — 7 Services

ZooKeeper	— Kafka coordination
Kafka	— event streaming broker
Cassandra	— NoSQL time-series store
Hadoop NameNode	— HDFS metadata
Hadoop DataNode	— HDFS block storage
Grafana	— live dashboard & visualisation
Prometheus	— infrastructure monitoring

Observability & Visualisation

- Grafana auto-provisioned via YAML — datasource and both dashboards load at container start
- 30-second auto-refresh — queries Cassandra directly, no intermediate API or cache
- Log-scale dashboard: base-10 Y-axis enables fair volatility comparison across all 6 assets
  - Without log scale: BTC ($80k) dominates; ADA ($0.40) is invisible
- Anomaly feed panel — filters Cassandra for is_anomaly = True rows in real time
- Prometheus scrapes all infrastructure components for health metrics
- Alert feed in production: anomaly rows would trigger Slack / PagerDuty notifications

30s	1.5%	Log_{10}
Dashboard refresh	Anomaly threshold	Y-axis scale

Conclusion

End-to-End in Real Time

WebSocket ⟶ Kafka ⟶ Spark ⟶ Cassandra ⟶ Grafana. Raw Binance events become a live dashboard in under 2 minutes, with fault-tolerant reconnection built in.

Streaming-Safe Design

Watermarked tumbling windows, stream-static HDFS join, and boolean anomaly flags — handles late data and bad messages gracefully without unbounded memory growth.

One-Command Deploy

Seven containerised services on a shared bridge network. Clone the repo, run start_pipeline.sh, and the live dashboard appears — zero manual configuration.

Thank you — Questions?